

Uart_Esp32 - Projektdokumentation - Firmware

Decoder-Anpassung an st10 | Version v0.12

Angabe	Wert
Projektname	Uart_Esp32
Dokumentenart	Projektdokumentation
Thema	Firmware fuer ESP32-WROOM-32 / NodeMCU-32S - UART-Decoder nach st10
Erstellungsdatum	2026-09-17
Autor / verantwortliche Person	OpenAI ChatGPT (technische Erstellung)
Version	v0.12
Status	Entwurf / lokaler PlatformIO- und Hardwaretest ausstehend
Aenderungsdatum	2026-09-17
Ablageort	Projektordner `Uart_ESP32/docs`
Referenzen	`Regeln_Projektdokumentation_PDF_DOCX_Pflicht_(5).pdf`; `ESP32_Protokoll_Decoder_st10.pdf`; Projektstand v0.11

Aenderungshistorie

Version	Datum	Bearbeiter	Status	Aenderung
v0.10	2026-09-17	OpenAI ChatGPT	Entwurf	UART START/STOP ergaenzt
v0.11	2026-09-17	OpenAI ChatGPT	Entwurf	Feld-5-TX-Replay (-0,5/+0,5, ca. 1 s/50 Hz) ergaenzt
v0.12	2026-09-17	OpenAI ChatGPT	Entwurf	Decoder auf st10- Outer-/Inner-Frame und Commands 0x0021/0x0023/0x0 031/0x0033 umgestellt

Ziel des Standes v0.12

Der bisherige Decoder erkannte das Hauptpaket im Wesentlichen ueber Route/Typ 0x10, OuterLength 0x15 und feste Bytepositionen. Der neue st10-Stand beschreibt das Protokoll genauer als verschachtelten Outer-/Inner-Frame. v0.12 passt den Decoder deshalb an diese belastbarere Struktur an.

Der Raw-/Sniffer-Modus bleibt unveraendert. Die UART-Hardwareparameter, Feld-Aliase, Kalibrierung, NVS-Persistenz, START/STOP-Steuerung, MQTT und Weboberflaeche bleiben erhalten.

st10-Protokollmodell

Outer Frame

FF FB Route OuterLength [InnerFrame mit OuterLength Bytes]

Gesamtlaenge:

$\text{totalFrameLength} = 4 + \text{OuterLength}$

Inner Frame

FF FD/FE InnerLength_LE Command_LE DATA... XOR

Konsistenzpruefung:

$\text{OuterLength} == \text{InnerLength} + 4$

$\text{DataLength} = \text{InnerLength} - 3$

XOR wird ueber InnerLength low, InnerLength high, beide Command-Bytes und alle DATA-Bytes gebildet. Outer-Header FF FB und innerer Marker FF FD/FE gehen nicht in die XOR-Berechnung ein.

Geaenderte Decoderlogik

Ein Frame gilt ab v0.12 nur dann als gueltig, wenn alle folgenden Bedingungen erfuehlt sind:

1. Synchronisation auf `FF FB`.
2. OuterLength ist plausibel und bestimmt die vollstaendige Framegroesse.
3. Innerer Marker ist `FF FD` oder `FF FE`.
4. InnerLength wird als `uint16 little endian` gelesen.
5. `InnerLength + 4 == OuterLength`.
6. Command wird als `uint16 little endian` gelesen.

7. `DataLength = InnerLength - 3` passt exakt zur Framengrösse.
8. XOR ueber den dokumentierten st10-Bereich stimmt mit dem letzten Byte ueberein.
9. Erst danach wird der Command-spezifische Decoder aufgerufen.

Bei unplaesiblen Frames wird nicht interpretiert; der Parser sucht anschliessend erneut nach FF FB.

Bekannte Commands

Command	Erwartete DataLength	Umsetzung v0.12
`0x0021`	14	Hauptdatensatz: 2 Meta-Bytes + 6 x uint16 LE; Feld 1..5 normierbar, Feld 6 Rohwert
`0x0023`	11	V1 = uint16 LE, V2 = uint8, V3 = uint16 LE; restliche Semantik offen
`0x0031`	3	uint16 LE + Statusbyte
`0x0033`	14	2 Meta-Bytes + 6 x uint16 LE; Werte separat dargestellt, semantische Reihenfolge offen

Unbekannte Commands werden gezaehlt, aber nicht semantisch interpretiert.

Command 0x0021 und Feld 1..6

Fuer 0x0021 besteht DATA aus 14 Bytes:

```
DATA[0]      Meta0
DATA[1]      Meta1
DATA[2..3]   Feld 1 uint16 LE
DATA[4..5]   Feld 2 uint16 LE
DATA[6..7]   Feld 3 uint16 LE
DATA[8..9]   Feld 4 uint16 LE
DATA[10..11] Feld 5 uint16 LE
DATA[12..13] Feld 6 uint16 LE
```

Die bisherigen technischen Namen Feld 1 bis Feld 6 bleiben erhalten. Benutzer-Aliase werden weiterhin nur zusaetzlich angezeigt. Die vorhandene Min/Mitte/Max-Normierung wird ausschliesslich auf Feld 1 bis Feld 5 von 0x0021 angewendet.

Webmonitor

Der UART-Monitor zeigt ab v0.12 zusaetzlich:

- letzte Route,
- Inner-Typ `FD`/`FE`,
- InnerLength,
- letzten Command,
- DataLength,
- getrennte Zaehler fuer `0x0021`, `0x0023`, `0x0031`, `0x0033`,
- die bekannten Teilwerte von `0x0023` und `0x0031`,
- die sechs strukturell decodierten Werte von `0x0033`.

Die Tabelle Feld 1..6 ist nun explizit als Daten des Commands 0x0021 gekennzeichnet.

MQTT

Unter <Basis>/uart/ werden neben den bisherigen Topics zusaetzlich ausgegeben:

- `last\_route`
- `last\_inner\_type`
- `last\_command`
- `last\_data\_length`
- `cmd0021\_count`
- `cmd0023\_count`
- `cmd0031\_count`
- `cmd0033\_count`

Die vorhandenen field1..field6-Topics beziehen sich weiterhin auf den Hauptdatensatz 0x0021.

TX-Replay und Abgrenzung

Der bereits vorhandene Feld-5-TX-Replay bleibt technisch erhalten und verwendet ab v0.12 explizit das zuletzt gueltige 0x0021-Frame als Vorlage. Feld 5 liegt dort weiterhin an derselben DATA-Position; nach Aenderung wird die XOR-Pruefsumme nach der st10-Regel neu berechnet.

Wichtig: st10 belegt nur die Empfangsrichtung D2 (TX des Zielgeraets) zum ESP32-RX. Fuer D4/RX des Zielgeraets ist in st10 kein gueltiges Eingangstelegramm belegt. Der TX-Replay ist deshalb weiterhin als experimentelle Testfunktion gekennzeichnet und nicht als bestaetigtes Gegenrichtungsprotokoll zu verstehen.

Validierung

Die im st10-Dokument angegebene Beispielsequenz

```
FF FB 10 15 FF FD 11 00 21 00 00 00 D3 05 CF 05 D5 05 DE 05 DE 05 54 01 A9
```

wird mit der neuen Logik als 25-Byte-Frame erkannt:

- OuterLength `0x15 = 21`
- InnerLength `0x0011 = 17`
- `17 + 4 = 21`
- Command `0x0021`
- DataLength `14`
- XOR-Ergebnis `0xA9` = empfangene Pruefsumme

Die geaenderte UartManager.cpp wurde zusaetzlich mit einem lokalen C++17-Syntaxcheck gegen ESP32/Arduino-Stubs geprueft. Ein vollstaendiger PlatformIO-Build und Hardwaretest muessen lokal auf dem Zielsystem erfolgen.

Geaenderte Dateien v0.12

- `src/UartManager.h`
- `src/UartManager.cpp`
- `src/main.cpp`
- `src/ConfigDefaults.h`
- `README.md`
- `docs/2026-09-17 - Uart\_Esp32 - Projektdokumentation - Firmware - v0.12.md`
- `docs/2026-09-17 - Uart\_Esp32 - Projektdokumentation - Firmware - v0.12.docx`

- `docs/2026-09-17 - Uart\_Esp32 - Projektdokumentation - Firmware - v0.12.pdf`
- `docs/2026-09-17 - Uart\_Esp32 - Commit-Nachricht - st10-Decoder - v0.12.txt`